

SAMD-SRG: Service-Aware Microservice Deployment Using Spectral Ranking and Graph-Enhanced Reinforcement Learning

Keli Liu, Jing Yang*, *Member, IEEE*, Xiaoli Ruan, Han Zhao, Shixuan Sun, *Member, IEEE*, Shaobo Li, and Minyi Guo, *Fellow, IEEE*

Abstract—Edge computing is crucial for latency-sensitive applications, making microservice deployment in edge environments a significant area of focus. However, microservice deployment still faces challenges in handling the intricate dependencies among microservices, optimizing heterogeneous resources, and navigating the expansive state space arising from dynamic request patterns. To address these challenges, we propose SAMD-SRG, a novel microservice deployment framework that integrates spectral clustering and graph reinforcement learning. Specifically, we first employ SpecRank, a microservice ranking method utilizing Hermitian spectral clustering, to enable intelligent grouping and ordering. We then design Hierarchical Service Representation Learning (HSRL), leveraging graph convolutional networks and cross-layer multi-head attention to capture bidirectional service-edge interaction representations. Finally, we adopt Graph-Enhanced Actor-Critic (GEAC) to optimize microservice deployment decisions. Evaluations across four cross-regional scenarios show that SAMD-SRG significantly outperforms existing baselines, with 48% reduction in response time, 52.50% lower deployment costs, and 75.93% improved load balancing in benchmark Scene 1.

Index Terms—Cloud services, microservices, resource allocation, graph neural networks, reinforcement learning

I. INTRODUCTION

EDGE computing environments present unique challenges for microservice deployment due to their inherent system characteristics. As illustrated in Fig. 1, service requests originating from end-users across diverse applications exhibit dynamic variations in both spatial and temporal dimensions. These time-varying requests are routed through networks to geographically distributed edge server clusters for collaborative processing [1], [2]. However, edge servers, constrained by physical deployment environments and economic considerations, typically possess limited computational capabilities

This work was supported by the national natural science foundation of China (62441608), the science and technology project of Guizhou Province (QKHZC[2023]368, QKHCG[2025]007, QKHZC[2025]023), the developing objects and projects of scientific and technological talents in Guiyang city (No.ZKHT[2023]48-8), Guizhou University Science and Technology Innovation Team ([2024]No.07), Guizhou University Basic Research Fund ([2024]08). (Corresponding author: Jing Yang.)

Keli Liu, Jing Yang, Xiaoli Ruan and Shaobo Li are with the State Key Laboratory of Public Big Data, Guizhou University, Guiyang, 550025, China (e-mail: gs.kliu23@gzu.edu.cn; jyang23@gzu.edu.cn; xluan@gzu.edu.cn; lishaobo@gzu.edu.cn).

Jing Yang, Han Zhao, Shixuan Sun and Minyi Guo are with the Department of Computer Science and Engineering, Shanghai Jiao Tong University, Shanghai, 200240, China. (e-mail: jyang23@gzu.edu.cn; zhao-han@cs.sjtu.edu.cn; sun-sx@cs.sjtu.edu.cn; guo-my@cs.sjtu.edu.cn).

This work was developed by the IEEE Publication Technology Department.

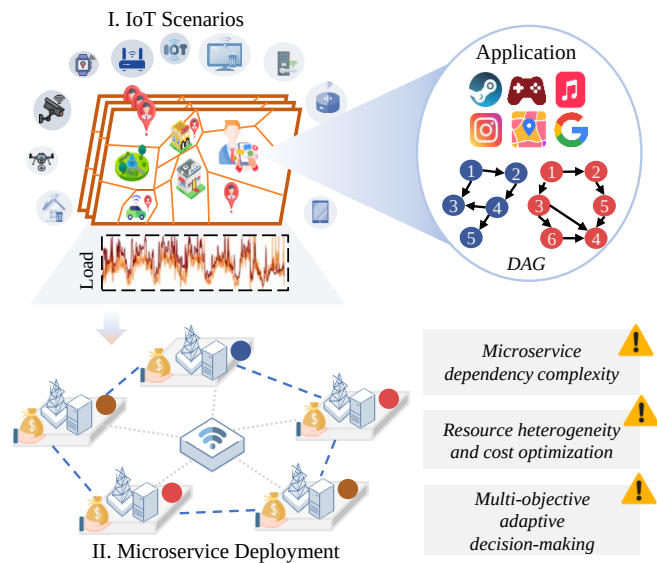


Fig. 1. Microservice deployment in Edge-cloud environments (where I. shows the Internet of Things (IoT) scenario generating variable workload patterns; II. demonstrates the deployment of microservices to distributed edge nodes.)

and heterogeneous resource configurations [3]. Consequently, these constraints significantly impact the efficacy of microservice deployment strategies. From an economic perspective of service providers, edge-cloud infrastructure generally adopts a resource usage-based billing model [4], where the scaling of microservice instances and resource occupation directly correlates with increased operational costs. Moreover, suboptimal microservice deployment strategies can lead to substantial service quality degradation, imbalanced computational resource allocation, and inefficient network bandwidth utilization.

Therefore, this research investigates in depth: How can a service-aware, resource-efficient, and adaptive microservice deployment framework be developed for edge-cloud environments, simultaneously optimizing Quality of Service (QoS) and resource utilization efficiency? However, achieving this objective faces the following key challenges:

(i) *Microservice dependency complexity and critical service awareness.* Modern applications are typically decomposed into tens or even hundreds of microservice components, forming high-dimensional invocation dependency networks [5]. Traditional approaches struggle to accurately identify critical service combinations and capture the affinity between

microservices, leading to suboptimal deployments. How can we precisely identify and prioritize critical path services in a combinatorially explosive dependency space [6], [7]?

(ii) *Heterogeneous resource representation and parallel cost optimization.* Edge-cloud environments are characterized by multi-dimensional resource heterogeneity and distributed deployment constraints, accompanied by diverse resource pricing models from various edge service providers. Existing studies frequently adopt simplified resource models [8], lacking systematic context abstraction and analysis of the coupling relationship between microservice deployment decisions. How can we achieve fine-grained resource allocation while optimizing cost-effectiveness in parallel?

(iii) *Multi-objective adaptive decision-making under high-dimensional state spaces.* Real-world user requests exhibit substantial temporal and spatial variability, while the invocation frequency and resource requirements among microservices demonstrate significant heterogeneity. Traditional methods incur excessive network resource consumption during voluminous state data transmission, and prove inadequate for efficient processing of high-dimensional state and action spaces. How can we construct a microservice deployment framework capable of both efficient key information extraction and decision space dimensionality reduction?

Based on the aforementioned requirements and challenges, we propose SAMD-SRG, an intelligent microservice deployment framework that integrates spectral clustering and graph reinforcement learning. This framework optimizes response time, deployment costs, and load balancing through a multi-objective reward function, enabling precise modeling of microservice dependencies, fine-grained optimization of resource utilization, and adaptive decision-making in dynamic environments. The main contributions are summarized as follows:

- To effectively reduce cross-node communication overhead, we propose SpecRank, a spectral clustering-based microservice ranking method. This approach utilizes a Hermitian adjacency matrix to encode asymmetric invocation relationships, combined with spectral decomposition and centrality optimization, achieving intelligent microservice grouping and hierarchical ordering.
- To address complex topological dependencies and resource constraints in heterogeneous edge-cloud environments, we introduce Hierarchical Service Representation Learning (HSRL). This method leverages residual graph convolutional networks to learn features from both infrastructure and service layers, and employs a cross-layer multi-head attention mechanism to model bidirectional service-edge relationships. The Service-Edge Compatibility Assessment (SECA) module then utilizes this information to guide resource-aware deployment decisions.
- We formalize the microservice deployment problem as a Markov Decision Process and design Graph-Enhanced Actor-Critic (GEAC), a method employing shared graph representation learning. Furthermore, we optimize state representation through Gumbel-Softmax reparameterization and enhance the stability of policy updates by incorporating KL divergence regularization.
- Comprehensive evaluations across four cross-regional

scenarios demonstrate that our approach significantly outperforms existing baselines. Notably, in benchmark Scene 1, SAMD-SRG achieves up to a 48% reduction in average response time, a 52.50% decrease in microservice deployment costs, and a 75.93% improvement in load balancing.

The remainder of the article is organized as follows: Section II analyzes related work. Section III describes the system model and elaborates on problem definition. Sections IV through VI detail the SpecRank, HSRL, and GEAC, respectively. Section VII provides a comprehensive performance evaluation. Finally, Section VIII concludes the article and discusses directions for future research.

II. RELATED WORKS

A. Microservice Dependency Modeling

Microservice architecture facilitates fine-grained resource allocation and flexible scheduling by decomposing applications into loosely coupled service units. However, the intricate dependencies among microservices pose a significant challenge for service deployment. In recent years, researchers have proposed diverse modeling approaches to represent microservice dependencies [9], [10], [11], [12], [13]. Qian et al. [12] constructed an undirected graph to represent structural dependencies and business functional relationships of monolithic applications, where nodes represent microservices and edges represent invocation relationships. However, this approach disregards communication directionality and dependency strength. Hu et al. [11] further modeled microservice applications as Directed Acyclic Graph (DAG), focusing on efficiently scheduling and offloading interdependent microservices on edge devices. Liu et al. [9] adopted Dynamic Graph Neural Networks (DGNN) to capture the dynamic invocation relationships and temporal information between services, modeling service invocation sequences as dynamic graphs. Furthermore, microservice dependencies significantly influence resource scheduling strategies [10], [14], [15], [16]. Zhang et al. [14] utilized Convolutional Neural Networks (CNNs) to identify dependency patterns among microservices and employed a two-stage prediction model to optimize resource allocation. He et al. [15] defined systems with complex dependencies and multi-instance deployment as a Service Placement Problem with Multiple microServices (SPPMS), proposing an efficient approximation solution employing greedy algorithms.

However, the aforementioned studies primarily adopt static representation mechanisms, which struggle to effectively capture the dynamic evolutionary characteristics of microservice dependencies under time-varying workload conditions. Moreover, existing research often focuses on localized performance optimization, neglecting the prioritization of critical services in resource scheduling and geographical allocation decision-making processes.

B. Dynamic Resource Allocation and Cost Optimization

Recent research has explored various approaches to address resource allocation in edge-cloud environments [8], [11], [22], [23], [17], [18], [24], [25], [26]. Ali et al. [25] proposed a

TABLE I
SUMMARY OF RELEVANT WORKS

MD = MICROSERVICE DEPENDENCIES; DM = DEPENDENCY MODELING; FE = FEATURE EXTRACTION; SP = SERVICE PRIORITY; L = LATENCY; LB = LOAD BALANCING; DC = DEPLOYMENT COST.

Ref	Problem	Algorithm	MD		SP	Metrics			Limitations
			DM	FE		L	LB	DC	
[1]	Joint optimization of microservice placement and load distribution	PPO	✓	×	×	✓	×	✓	• Static deployment environment limitation • Lack of dependency strength differentiation • Simplified load distribution algorithm
[4]	Microservice placement	Branch-and-bound	✓	×	×	✓	×	✓	• Idealized task queue model • Lack of dependency strength differentiation • Static deployment strategy
[8]	Microservice deployment	Neural network learning (Adam)	✓	×	×	✓	✓	✓	• Lack of dependency strength differentiation • Ignoring network transmission costs • Load imbalance in weighted round-robin
[10]	Microservice placement	DRL	✓	✓	×	✓	×	✓	• Simplistic user behavior patterns • Limitations in service conflict modeling
[13]	Dependency-aware microservice deployment	Priority-based Greedy Algorithm	✓	×	✓	✓	×	×	• High time complexity of the algorithm • Static deployment strategy
[14]	Dependent application offloading	Machine Learning	-	✓	×	✓	×	×	• Focuses solely on resource allocation
[15]	QoS-Aware Resource Management	Greedy	✓	×	×	✓	×	×	• Single optimization objective • Static deployment strategy
[17]	Multi-objective optimization	Genetic Algorithm (NSGA-III)	✓	×	×	✓	×	✓	• Ignoring microservice heterogeneity • Ignoring user request burstiness • Lack of dependency strength differentiation
[18]	Microservice composition optimization	Dueling DQN	✓	×	×	✓	✓	✓	• Resource over-provisioning results in low utilization • Long initial training time
[19]	Service optimization	Hybrid-method-based iterative algorithm	×	×	×	✓	×	×	• Single optimization objective • Ignoring microservice dependencies
[20]	Microservice deployment	DDPG	×	×	×	✓	×	×	• Single optimization objective • Ignoring microservice dependencies
[21]	Microservices-based application placement	Q-learning	✓	×	×	✓	×	×	• Static deployment environment limitation • Assume one request per time slot
Our work	Service-aware microservice deployment	Spectral Ranking and Graph Reinforcement Learning	✓	✓	✓	✓	✓	✓	• Addresses the limitations mentioned above • Comprehensive multi-objective optimization

hybrid multi-objective optimization algorithm based on Chaos Theory, which effectively reduces the execution cost of workflow tasks in cloud data centers. Tang et al. [8], [17] investigated cost-aware microservice deployment methods for IoT applications, optimizing deployment strategies to minimize resource consumption costs. However, this approach neglects the impact of network transmission costs. Hu et al. [11] employed a Priced Timed Automata model to describe how task communication and execution evolve resource costs over time. In their model, resource consumption costs are primarily related to task execution time, processor performance, and cost growth rates, yet fail to comprehensively reflect the actual cost composition in edge-cloud environments. Addressing resource dynamics in edge environments, Fu et al. [18] proposed the Nautilus, which dynamically adjusts microservice deployment and resource allocation to adapt to load variations and resource competition, minimizing computational resource usage while maintaining QoS. To mitigate increased deployment costs due to frequently changing resource requirements and the allocation of new and existing microservices, Zambianco et al. [24] introduced an interruption-aware microservice reorchestration algorithm that maximizes cost-effectiveness in multi-cloud deployments by rescheduling microservices from high-cost nodes to the cheapest available nodes.

However, existing resource allocation research inadequately characterizes the dynamic properties of multi-dimensional heterogeneous resources, hindering the collaborative optimization of resource utility maximization and deployment cost minimization. Therefore, there is an urgent need to achieve fine-grained resource allocation and cost optimization.

C. Multi-Objective Collaborative Microservice Deployment

Microservice deployment is a complex optimization problem that can be decomposed into resource allocation, load balancing, and QoS challenges. Researchers have proposed various approximation-based optimization methods for microservice deployment [17], [27], [19], [28], [29], [30], addressing service placement and cost optimization issues. However, these approaches often converge to local optima and struggle to meet real-time decision-making requirements in large-scale edge environments. An alternative research direction has focused on utilizing Reinforcement Learning (RL) methods to solve task offloading optimization strategies [31], [32], [20], [33], [34], [35], [36], [37], [38]. For instance, Moghaddasi et al. [37] proposed a task offloading method based on Deep Reinforcement Learning (DRL), which not only significantly enhances the performance of terminal devices but also effectively optimizes the utilization of both edge

and cloud computing resources. Gao et al. [38] proposed the Com-DDPG multi-agent reinforcement learning framework, aiming to optimize task offloading decisions and address issues of task dependency and resource competition. In recent years, Graph Neural Networks (GNNs) have been increasingly applied to microservice dependency analysis and deployment optimization [10], [39], [40], [41]. For example, Chy et al. [40] proposed a machine learning approach based on GNNs and Variational Autoencoders (VAE) for optimizing microservice decomposition and layout. GNNs naturally represent microservice topological structures and learn implicit features of nodes and edges through message-passing mechanisms, providing new insights for dependency-aware deployment decisions.

However, current microservice deployment research faces two key challenges: (i) existing methods predominantly focus on single optimization objectives (such as latency, cost, or resource utilization), lacking collaborative optimization of multiple objectives; (ii) current feature representation methods fail to differentiate the varying impacts of dynamic dependency characteristics, such as inter-service invocation frequency, on deployment decisions. Therefore, there is an urgent need to construct a unified framework that integrates graph structure representation learning with decision optimization. Table I shows the comparison with existing deployment approaches.

III. SYSTEM MODEL AND PROBLEM STATEMENT

The architecture overview, latency model, cost model, load balancing model and problem formulation are defined in this section.

A. Architecture Overview

We construct a heterogeneous edge-cloud environment consisting of N edge nodes and one central cloud server. As illustrated in Fig. 1, the system comprises the following components:

- **Edge Servers.** Each edge server $e_i \in E = \{e_1, e_2, \dots, e_N\}$ is equipped with limited computing resources, denoted by $C_i = \{c_i^{CPU}, c_i^{MEM}\}$, representing CPU computing capacity and Memory capacity, respectively. Each edge node covers a specific geographical area with a service radius R_e^i and geographical location $L_i = (l_i^x, l_i^y)$. Edge nodes are interconnected through a network, with the bandwidth matrix defined as $B = [b_{ij}]_{N \times (N+1)}$, where b_{ij} represents the bandwidth capacity from node e_i to node e_j , and the last column of the matrix indicates the connection bandwidth between nodes and the cloud server.

- **Cloud Server.** Let C denote the central cloud server, which is responsible for processing service requests that edge nodes cannot accommodate. The cloud server possesses abundant computing and storage resources but exhibits higher communication latency compared to edge nodes.

- **User Requests.** Let $U = \{u_1, u_2, \dots, u_M\}$ denote the set of mobile users, where each user's geographical location L_i^t changes dynamically over time. The time domain is discretized into 24 time periods $T = \{t_1, t_2, \dots, t_{24}\}$, corresponding to the 24 hours of a day. Within each time period t , user requests

R_t follow specific temporal distribution patterns, reflecting typical daily activity characteristics.

- **Applications and Microservices.** Applications consist of interdependent microservices, with different types of applications $A = \{a_1, a_2, \dots, a_k\}$ containing varying numbers of microservices. Each application a_k comprises n_k microservices $\{s_1^k, s_2^k, \dots, s_{n_k}^k\}$. Each microservice $s_i \in S$ has resource requirements $r_i = [r_i^{CPU}, r_i^{MEM}]$. The affinity relationship between microservices is represented by $\Phi_k = [a_{ij}^k]$, where a_{ij}^k quantifies the interaction intensity between microservices s_i and s_j in application a_k . Higher affinity indicates greater communication frequency or data transfer volume between microservices.

Upon user service request initiation, the geographically proximate edge node is designated as the initial access point. The requested application is then decomposed into its constituent microservices. For the entry microservice, local processing occurs if the deployment node possesses adequate resources; otherwise, the task is offloaded to the cloud server. Subsequent microservices are routed along the optimal execution path, determined by deployment status. The notation used is summarized in Table II.

B. Latency Model

In edge-cloud environments, user devices transmit service requests to edge servers through wireless communication protocols (e.g., ZigBee and Bluetooth Low Energy (BLE)). For a request initiated by user $u_j \in U$ at time t invoking application a_k , the total response latency can be categorized into three deployment scenarios based on microservice placement:

1) **Edge Server Deployment:** When all microservices are deployed on resource-sufficient edge servers, the total response latency is:

$$T_{tot}^r(t) = T_{up}^e(t) + \sum_{s \in S_a} T_{proc}^s(t) + \sum_{i=1}^{n-1} T_{com}^{(s_i, s_{i+1})}(t) + T_{down}^e(t) \quad (1)$$

where T_{up}^e and T_{down}^e represent uplink and downlink communication latency between users and edge nodes, $T_{proc}^s = D_i/E_{proc}$ denotes processing latency for all microservices on edge nodes, and $T_{com}^{(s_i, s_{i+1})} = D_{(s_i, s_{i+1})}/B_{(e(s_i), e(s_{i+1}))}$ indicates inter-microservice communication latency (zero when deployed on the same node).

2) **Cloud Server Deployment:** When edge node resources cannot satisfy the resource requirements of the first microservice, all microservices of the application are processed in the cloud, resulting in a total response latency of:

$$T_{tot}^r(t) = T_{up}^c(t) + \sum_{s \in S_a} T_{proc}^s(t) + T_{down}^c(t) \quad (2)$$

where T_{up}^c and T_{down}^c represent uplink and downlink communication latency between users and cloud servers, and T_{proc}^s denotes processing latency for all microservices in the cloud.

3) **Edge-Cloud Collaborative Deployment:** When the first $m-1$ microservices are processed on edge servers, but the resource requirements of microservice s_m exceed available

TABLE II
NOTATIONS

Symbol	Description	Symbol	Description
N	Number of edge nodes	D_{s_i}	Data transmission volume of microservice s_i
C	The central cloud server	L_i^t	Geographical location of user u_i at time t
U	Set of mobile users	R_e^t	Service radius of edge node e_i
T	Set of time periods	L_j^t	Resource utilization rate of edge node e_j at time t
A	Set of applications	$\bar{L}$	Average load across all edge nodes
S	Set of all microservices	V_L	Load balancing degree of all edge nodes
b_{ij}	Bandwidth between edge nodes and to the cloud server	T_{avg}	Average response time of all user requests
R_t	User requests at time period t	C_{avg}	Average deployment cost of all user requests
C_e, C_c	Deployment cost for all microservices processed at edge nodes / cloud	C_e^{net}	Unit cost of data transmission at edge nodes
T_{tot}^r	The total response latency	C_c^{net}	Unit cost of data transmission in the cloud
T_{proc}^s	Processing latency for all microservices	$T_{com}^{(e,c)}$	Communication latency from edge to cloud
$T_{com}^{(s_i, s_{i+1})}$	Communication latency between microservices s_i and s_{i+1}	$S_{r,e}, S_{r,c}$	Set of microservices in request r processed at the edge / cloud
T_{up}^e, T_{down}^e	Uplink / Downlink communication latency between users and edge nodes	$L_i = (l_o^i, l_a^i)$	Geographical location of edge node e_i
T_{up}^c, T_{down}^c	Uplink / Downlink communication latency between users and cloud server	$\Phi_k = [af_{ij}^k]$	Affinity matrix between microservices in application a_k
c_e^{CPU}, c_e^{MEM}	CPU / Memory capacity of edge server e_i	r_i^{CPU}, r_i^{MEM}	CPU / Memory resource requirements of microservice s_i
c_e^{CPU}, c_e^{MEM}	Unit costs of CPU / Memory resources at edge nodes	c_c^{CPU}, c_c^{MEM}	Unit costs of CPU / Memory resources in cloud
		$E = \{e_1, \dots, e_N\}$	Set of edge nodes

edge node resources, it is deployed in the cloud, resulting in a total response latency of:

$$T_{tot}^r(t) = T_{up}^e(t) + \sum_{i=1}^{m-1} T_{proc}^e(t) + \sum_{i=1}^{m-2} T_{com}^{(s_i, s_{i+1})}(t) + T_{com}^{(e,c)}(t) + \sum_{i=m}^n T_{proc}^c + T_{down}^c \quad (3)$$

where $T_{com}^{(e,c)}$ represents communication latency from edge to cloud. The average response time of all user requests is defined as $T_{avg} = \frac{1}{j} \sum_{j=1}^j T_{tot}^r$, with response time threshold L_{th} serving as a hard constraint for QoS guarantees.

C. Cost Model

The deployment cost is derived from the resource utilization and network transmission costs incurred by microservice processing for all user requests.

1) *Edge Server Deployment Cost*: The deployment cost C_e for all microservices processed at edge nodes is calculated as:

$$C_e = \sum_{t \in T} \sum_{r \in R(t)} \sum_{s \in S_{r,e}} (c_e^{CPU} r_{s_i}^{CPU} + c_e^{MEM} r_{s_i}^{MEM} + 2C_e^{net} D_{s_i}) \quad (4)$$

where $S_{r,e}$ represents the set of microservices in request r processed at the edge, c_e^{CPU} and c_e^{MEM} denote the unit costs of CPU and Memory resources at edge nodes respectively, C_e^{net} is the unit cost of data transmission at edge nodes, and D_{s_i} represents the data transmission volume of the microservice.

2) *Cloud Server Deployment Cost*: The deployment cost C_c for all microservices offloaded to the cloud is formulated as:

$$C_c = \sum_{t \in T} \sum_{r \in R(t)} \sum_{s \in S_{r,c}} (c_c^{CPU} r_{s_i}^{CPU} + c_c^{MEM} r_{s_i}^{MEM} + 2C_c^{net} D_{s_i}) \quad (5)$$

where $S_{r,c}$ represents the set of microservices in request r processed in the cloud, c_c^{CPU} and c_c^{MEM} denote the unit costs of CPU and Memory resources in cloud servers respectively, and C_c^{net} is the unit cost of data transmission in the cloud.

The average deployment cost $C_{avg} = \frac{1}{j} \sum_{j=1}^j (C_e + C_c)$ represents the mean cost consumed by all user requests, comprehensively considering the resource consumption costs of each request on both edge servers and cloud servers.

D. Load Balancing Model

The load balancing model monitors the resource utilization rate L_j^t of edge node e_j to effectively prevent microservice performance degradation caused by resource overload during request traffic peaks:

$$L_j^t = \eta \cdot \left(1 - \frac{C_{CPU}^r}{C_{CPU}^{max}}\right) + (1 - \eta) \cdot \left(1 - \frac{C_{MEM}^r}{C_{MEM}^{max}}\right) \quad (6)$$

where η is a weighting coefficient. C_{CPU}^r and C_{MEM}^r represent the available CPU and Memory capacity of the current edge node, while C_{CPU}^{max} and C_{MEM}^{max} denote the maximum CPU and Memory capacity across all edge nodes. The load balancing degree of all edge nodes is measured through load variance:

$$V_L(t) = \frac{1}{N} \sum_{j=1}^N (L_j^t - \bar{L}(t))^2 \quad (7)$$

where $\bar{L}(t) = \frac{1}{N} \sum_{i=1}^N L_j^t$ is the average load across all edge nodes. A smaller load variance indicates more balanced resource allocation among multiple edge servers.

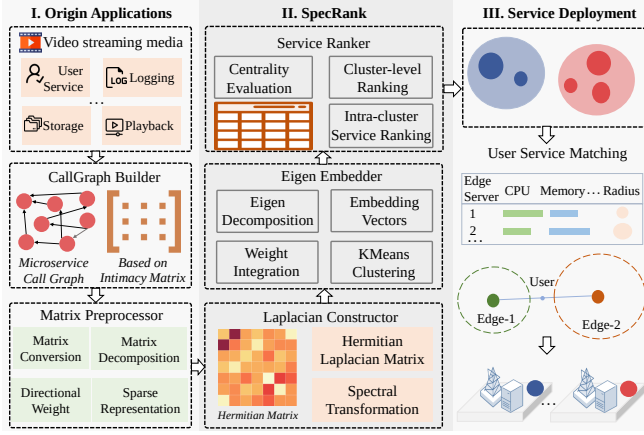


Fig. 2. SpeRank Framework (where I. illustrates the construction of microservice application call graphs; II. presents the spectral clustering-based service ranking method; III. depicts the service deployment leveraging SpecRank, which places closely related microservices on adjacent nodes.)

E. Problem Formulation

Based on the system model, we formalize the microservice deployment optimization problem as a multi-objective constrained optimization problem. The optimization objective is to minimize the following weighted multi-objective function:

$$\min J(t) = \omega_1 \cdot T_{avg} + \omega_2 \cdot C_{avg} + \omega_3 \cdot V_L \quad (8)$$

$$\text{s.t.} \begin{cases} \sum_{s_i \in S_j} r_{s_i}^{CPU} \leq C_j^{CPU} \\ \sum_{s_i \in S_j} r_{s_i}^{MEM} \leq C_j^{MEM} \\ 0 \leq L_j^t \leq 1, 0 \leq V_L \leq 1 \end{cases} \quad (9)$$

We prove this problem is NP-hard through polynomial-time reduction from the Multi-dimensional Bin Packing Problem. The complexity arises from the exponential decision space (N^S possible deployment schemes), competing objectives, dynamic request patterns, and nonlinear objective components. Our solution employs a two-stage approach: spectral clustering for microservice grouping, followed by Graph-Enhanced microservice deployment strategies.

IV. SPECTRAL CLUSTERING-BASED MICROSERVICE RANKING

To simultaneously reduce cross-node communication overhead and enable effective identification of critical services in microservice deployment, we introduce **SpecRank**, a microservice ranking method (Fig. 2).

Microservice dependencies are modeled as a weighted directed graph $G = (M, E, A)$. Traditional spectral clustering methods typically apply symmetrization processing to the adjacency matrix or completely ignore directionality, leading to the loss of critical hierarchical information in microservice call chains. Based on the mathematical principle that any real matrix can be uniquely decomposed into symmetric and antisymmetric parts, we employ a Hermitian adjacency matrix to preserve directional information:

$$H(t) = S(t) + iA_A(t) \quad (10)$$

Algorithm 1: SpecRank

Input: Weighted directed graph $G = (M, E, A)$ representing microservice dependencies, number of clusters K , and weight factors $\lambda_1, \lambda_2, \lambda_3$.
Output: Optimized deployment order for microservices.

- 1: **Hermitian Spectral Clustering:**
- 2: Compute the Hermitian adjacency matrix using Eq. (10):
- 3: Calculate the out-degree matrix D_{out} for normalization.
- 4: Compute the normalized Hermitian Laplacian using Eq. (11):
- 5: Apply Lanczos method for eigendecomposition using Eq. (12):
- 6: Select eigenvectors corresponding to the K smallest eigenvalues.
- 7: Embed microservices into K -dimensional feature space.
- 8: Apply K-means clustering to obtain microservice clusters $\{C_1, C_2, \dots, C_K\}$.
- 9: **Microservice Ranking:**
- 10: Initialize empty list $\mathcal{O}$ for final deployment order.
- 11: **for each cluster C_j do**
- 12: **for each microservice $S_i \in C_j$ do**
- 13: Compute PageRank value $PR(S_i)$ using Eq. (14).
- 14: Calculate microservice ranking score $C(S_i)$ using Eq. (13):
- 15: Sort microservices in C_j by $C(S_i)$ in descending order to obtain $\mathcal{O}_{C_j}$.
- 16: Append sorted microservices $\mathcal{O}_{C_j}$ to $\mathcal{O}$.
- 17: **return** deployment order of all microservices

where $S(t) = (A(t) + A^T(t))/2$ captures bidirectional call intensity to identify tightly coupled service groups, and $A_A(t) = (A(t) - A^T(t))/2$ encodes directional weights to determine deployment priorities. This decomposition guarantees the Hermitian property $H^*(t) = H(t)$, ensuring mathematical stability of clustering results.

To enhance clustering stability, we compute the normalized Hermitian Laplacian matrix:

$$L_H(t) = I - D_{out}^{-1}(t)H(t) \quad (11)$$

where $D_{out}(t)$ is the out-degree matrix of microservices, normalizing different microservice call patterns; I is the identity matrix. Since $L_H(t)$ possesses Hermitian properties, we utilize the Lanczos method for eigendecomposition to reduce computational complexity:

$$L_H(t)u = \lambda u \quad (12)$$

where u_i represents the clustering embedding of microservice s_i ; λ_i reflects the clustering density of microservices, with smaller eigenvalues indicating microservices are more likely to belong to the same cluster. Selecting eigenvectors corresponding to the K smallest eigenvalues allows embedding microservices into a low-dimensional feature space, facilitating K-means clustering and minimizing cross-node communication overhead.

Following microservice clustering, the Service Ranker orders microservices within each cluster to determine the optimal deployment sequence. To measure microservice importance, we comprehensively consider call activity, call frequency, and PageRank importance, defining the microservice ranking score as follows:

$$C(S_i) = \lambda_1 \cdot D^{out}(S_i) + \lambda_2 \cdot D^{in}(S_i) + \lambda_3 \cdot PR(S_i) \quad (13)$$

where $D^{out}(S_i)$ represents the call activity level of microservice S_i ; $D^{in}(S_i)$ reflects the frequency at which S_i is called;

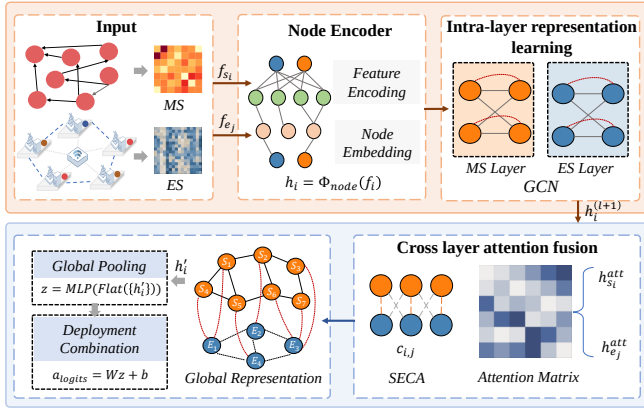


Fig. 3. HSRL method (where the upper side represents intra-layer representation learning; while the lower side depicts cross-layer attention fusion mechanism.)

$PR(S_i)$ is calculated as:

$$PR(S_i) = (1 - \alpha) + \alpha \sum_{j \in N(i)} \frac{PR(S_j)}{|N(j)|} \quad (14)$$

where α is the damping factor, and $N(i)$ represents the neighboring nodes of S_i . Finally, based on the ranking scores calculated from $C(S_i)$, microservices within each cluster are sorted in descending order, prioritizing the deployment of high-importance microservices to minimize core service response time. The main procedure of SpecRank is shown in Algorithm 1.

V. HIERARCHICAL SERVICE REPRESENTATION LEARNING

This section presents **HSRL**, a novel method for effectively capturing the complex topological dependencies and resource constraints in microservice systems. As shown in Fig. 3 and described in Algorithm 2, HSRL extracts feature representations from both the infrastructure layer (ES) and service layer (MS), providing comprehensive state inputs for subsequent deployment decisions.

A. Intra-Layer Representation Learning

In edge-cloud environments, the microservice deployment system is modeled as a hierarchical heterogeneous graph $G = (V, E)$, where the vertex set $V = V_s \cup V_e$ consists of service node set V_s and edge node set V_e . This heterogeneous graph describes inter-service dependencies and deployment constraints. The edge set E includes: (i) intra-service dependencies E_{ss} , (ii) inter-edge node connections E_{ee} , and (iii) microservice deployment relationships E_{se} .

A key challenge is transforming high-dimensional heterogeneous raw system states into feature vectors suitable for policy networks. Existing approaches suffer from two primary limitations: (i) high-dimensional features result in excessive policy network complexity; (ii) ineffective capture of graph structural information. To address this, we propose a deep representation embedding mechanism.

Algorithm 2: HSRL

Input: Hierarchical heterogeneous graph $G = (V, E)$, service node set V_s with features $\{f_{s_i}\}$, edge node set V_e with features $\{f_{e_j}\}$, neural network parameters Φ_{node} , W , b .
Output: Global representation z , service-edge deployment decision logits a_{logits} .

- 1: **Intra-Layer Representation Learning:**
- 2: **for each** node $i \in V_s \cup V_e$ **do**
- 3: Initialize node encoding h_i by Eq. (15).
- 4: **for each** service node $i \in V_s$ **do**
- 5: Update service representation by Eq. (16).
- 6: **for each** edge node $j \in V_e$ **do**
- 7: Update edge representation by Eq. (16).
- 8: **Cross-Layer Attention Fusion (CL):**
- 9: **for each** service node $i \in V_s$ **do**
- 10: Compute attention $h_{s_i}^{att}$ by Eq. (17).
- 11: **for each** edge node $j \in V_e$ **do**
- 12: Compute attention $h_{e_j}^{att}$ by Eq. (18).
- 13: **for each** service-edge pair (i, j) **do**
- 14: Calculate compatibility score $c_{i,j}$ by Eq. (19).
- 15: **Information Integration (II):**
- 16: **for each** node $i \in V_s \cup V_e$ **do**
- 17: Update embedding $h_i^{(l)}$ by Eq. (20).
- 18: Generate global representation $z = \text{MLP}(\text{Flat}(\{h_i^{(l)}\}))$.
- 19: Map to deployment logits $a_{logits} = W_{out}z + b_{out}$.
- 20: **return** z, a_{logits}

For microservice nodes $v_i^s \subseteq V_s$, we construct feature vector f_{s_i} containing computational resource demand $f_{s_i}^{CPU}$, Memory requirement $f_{s_i}^{MEM}$, and current deployment status $d_{s_i} = [d_{s_i, e_1}, d_{s_i, e_2}, \dots, d_{s_i, e_{N_e}}]$. For edge nodes $v_j^e \in V_e$, feature vector f_{e_j} includes CPU computing capacity $f_{e_j}^{CPU}$, Memory capacity $f_{e_j}^{MEM}$, bandwidth $f_{e_j}^B$, service radius $f_{e_j}^R$, and current resource utilization $f_{e_j}^U$. To map raw heterogeneous node features into a unified high-dimensional representation space, we design parameter-isolated deep node encoders Φ_{node} with shared architecture, employing a two-layer fully connected network structure combined with layer normalization and Rectified Linear Unit (RELU) activation:

$$h_i = \Phi_{node}(f_i) \quad (15)$$

To comprehensively capture intra-layer topological structures and node interaction characteristics, we develop a graph convolutional framework based on the message passing paradigm, performing deep feature extraction separately for the service layer and edge layer:

$$h_i^{(l+1)} = \sigma \left(\sum_{j \in N(i)} \frac{1}{\sqrt{d_i d_j}} W^{(l)} h_j^{(l)} + b^{(l)} \right) + h_i^{(l)} \quad (16)$$

where $\in N(i)$ denotes the neighbor set of node i , W represents the weight matrix, b indicates the bias term, and σ is the ReLU nonlinear activation function. Residual connections are incorporated after each graph convolution layer. This formulation, based on Graph Convolutional Networks (GCNs), is applied to both layers, respectively capturing service dependency topology and physical connection characteristics of edge nodes.

B. Cross-Layer Attention Fusion Mechanism

To address the dynamic matching problem between service demands and edge resource provisioning, we design a cross-layer information fusion mechanism based on multi-head attention. By considering service and edge node representations

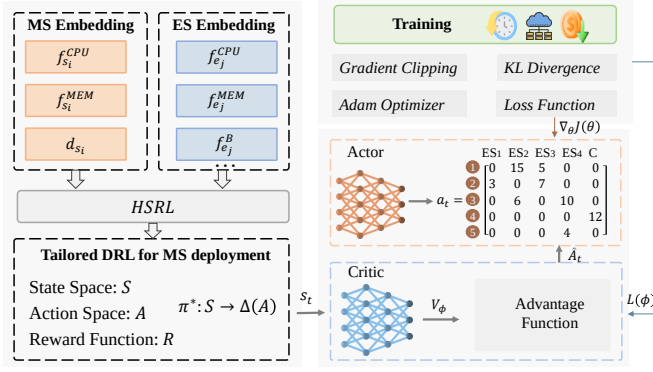


Fig. 4. GEAC Framework (where the left side shows the embedding process with HSRL integration and tailored DRL components; while the right side illustrates the training with Actor-Critic architecture for generating optimal deployment decisions.)

as sequential inputs, we employ bidirectional attention to: (i) enable each service to focus on the most suitable edge nodes for its requirements; and (ii) enable each edge node to attend to the most compatible services for its resource characteristics. The formal representation is as follows:

$$h_{s_i}^{att} = h_{s_i} + \text{MHA}(h_{s_i}, \{h_{e_j}\}, \{h_{e_j}\}) \quad (17)$$

$$h_{e_j}^{att} = h_{e_j} + \text{MHA}(h_{e_j}, \{h_{s_i}\}, \{h_{s_i}\}) \quad (18)$$

To comprehensively evaluate service-edge compatibility, we introduce the SECA, generating a service-edge matching matrix $c_{i,j}$ defined as:

$$c_{i,j} = f(h_{s_i}^{att} \odot h_{e_j}^{att}) \quad (19)$$

where $\odot$ denotes element-wise multiplication and f represents the compatibility evaluation function. $c_{i,j}$ is used to adjust final deployment decision probabilities.

Following cross-layer fusion, final information integration and decision generation are performed through Graph Attention Networks (GATs):

$$h'_i = \sigma \left(\sum_{j \in N(i)} \alpha_{ij} W h_j \right) \quad (20)$$

where $\alpha_{ij} = \frac{\exp(e_{ij})}{\sum_{k \in N(i)} \exp(e_{ik})}$ denotes the normalized attention coefficient between node i and its neighbor j , and h'_i represents the updated node embedding after graph attention aggregation.

The global system representation $z = \text{MLP}(\text{Flat}(\{h'_i\}))$ is obtained through a flattening operation and a multilayer perceptron. Finally, z is linearly mapped to the logits of the action space, i.e., $a_{\text{logits}} = W_{\text{out}}z + b_{\text{out}}$. The a_{logits} correspond to all possible service-edge node deployment combinations, providing the decision basis for the policy network.

VI. GRAPH-ENHANCED MICROSERVICE DEPLOYMENT

This section introduces **GEAC**, a graph reinforcement learning-based microservice deployment method (Fig. 4). The framework integrates SpecRank for spectral clustering-based microservice grouping and prioritization, utilizes HSRL for heterogeneous system topology feature extraction, and dynamically optimizes deployment through DRL.

Algorithm 3: GEAC

Input: Actor parameters θ , Critic parameters ϕ , initial state s_0 , maximum steps T
Output: Optimal policy π^*
1: Initialize Actor network π_θ , Critic network V_ϕ , and target Critic network $V_{\phi'}$ with $\phi' \leftarrow \phi$
2: Set initial state $s_t = s_0$
3: **for** $t = 0$ to T **do**
4: **Actor:**
5: Encode state s_t to obtain representation z , and a_{logits}
6: Compute π_θ using Eq. 23, and sample action a_t
7: Execute action a_t , observe next state s_{t+1} , and receive reward r_t
8: **Critic:**
9: Compute value function $V_\phi(s_t)$ using Eq. 26, and calculate target value $V^{\text{target}}(s_t)$
10: **Update Actor:**
11: Compute advantage function, policy gradient using Eq. 24, and Kullback-Leibler (KL) divergence using Eq. 25, update Actor parameters θ
12: **Update Critic:**
13: Compute value function loss using Eq. 27, apply gradient clipping, and update Critic parameters ϕ
14: Update target Critic parameters ϕ' periodically
15: Transition to the next state: $s_t \leftarrow s_{t+1}$
16: **end for**
17: **return** Optimal policy π^*

A. DRL Tailored for Microservice Deployment

We formulate the microservice deployment problem as a Markov Decision Process (MDP) defined by the tuple $M = (S, A, P, R)$, where:

- **State Space.** The system state $s_t \in S$ at time t is represented as a graph embedding, which includes the embedding vectors of all microservices and edge servers, along with their deployment relationships. These embeddings are extracted from the system topology and resource states using HSRL.

- **Action Space.** An action $a_t \in A$ at time t corresponds to deploying a specific microservice onto a particular edge server. The action space has a dimensionality of $|A| = N_s \times N_e$, where N_s and N_e denote the number of microservices and edge servers, respectively.

- **Transition Function.** $P(s_{t+1}|s_t, a_t)$ represents the probability of transitioning to state s_{t+1} after executing action a_t in state s_t . In this problem, environmental dynamics are determined by both microservice deployment decisions and user request patterns.

- **Reward Function.** $R(s_t, a_t)$ denotes the immediate reward obtained after executing action a_t in state s_t , which primarily consists of: (i) Response Time Reward R_T , (ii) Deployment Cost Reward R_C , (iii) Load Balancing Reward R_L . The composite reward is defined as:

$$R(s_t, a_t) = w_1 R_T(s_t, a_t) + w_2 R_C(s_t, a_t) + w_3 R_L(s_t, a_t) \quad (21)$$

Based on the MDP formulation, the optimization objective of microservice deployment is to find the optimal policy π^* that maximizes the expected cumulative discounted reward with entropy regularization:

$$J(\theta) = \mathbb{E}_{\pi_\theta} \left[\sum_{t=0}^{\infty} \gamma^t R(s_t, a_t) \right] + \alpha_H H(\pi_\theta) \quad (22)$$

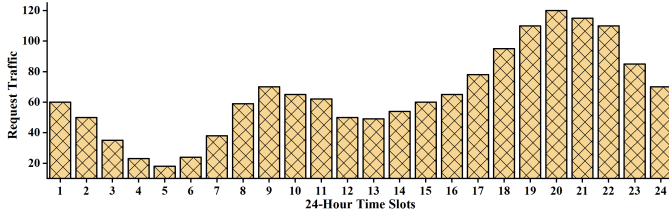


Fig. 5. Request Traffic Across 24-Hour Time Slots

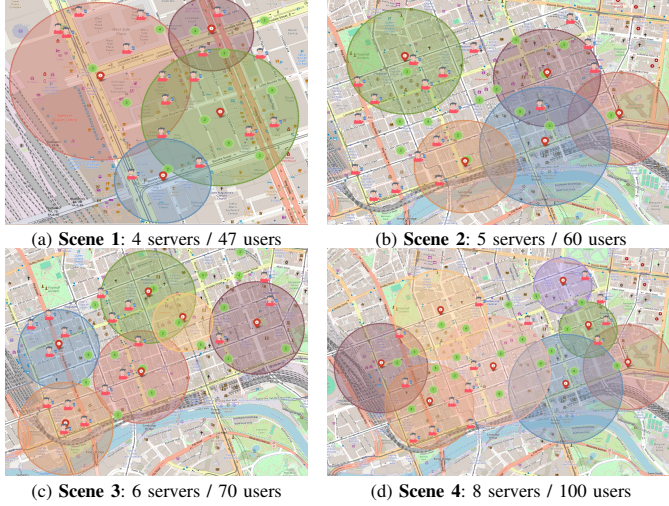


Fig. 6. Four cross-regional scenes

where $\gamma \in [0, 1)$ is the discount factor, balancing immediate and future rewards. $H(\pi_\theta) = -\sum_a \pi_\theta(a_t|s_t) \log \pi_\theta(a_t|s_t)$ is the policy entropy, encouraging exploration. α_H is a dynamically decaying regularization coefficient that balances exploration.

B. Graph-Enhanced Actor-Critic Method

GEAC employs a dual-network architecture comprising: (i) a policy network (Actor) that generates deployment decisions; and (ii) a value network (Critic) that evaluates system states. These networks share a unified graph representation learning module that extracts topological features, while maintaining functionally specialized output layers. Algorithm 3 illustrates the core process of GEAC.

1) *Actor for Deployment Decision-Making*: The policy network generates deployment decisions by producing action logits a_{logits} from the global feature representation encoded by the graph structure encoder. Considering resource constraints and service dependencies in microservice deployment environments, we integrate the service-edge matching matrix $c_{(i,j)}$ generated by SECA into the decision-making process, forming a soft constraint mechanism adapted to the dynamic characteristics of edge computing environments. For high-dimensional discrete action spaces, we employ the Gumbel-Softmax reparameterization technique to enable differentiable sampling and support gradient propagation. The action probability distribution is calculated as follows:

$$\pi_\theta(a_t|s_t) = \text{softmax}\left(\frac{a_{logits} + g}{\tau}\right) \quad (23)$$

TABLE III
CONFIGURATIONS OF EDGE NODE

Edge Node	CPU (MB)	Memory (GB)	CPU Cost (\$)	Memory Cost (\$)	Radius (m)
Node 1	1800	1300	0.05	0.03	150
Node 2	600	900	0.04	0.02	70
Node 3	1500	1800	0.06	0.04	130
Node 4	1400	1300	0.05	0.03	80

TABLE IV
MICROSERVICE RESOURCE REQUIREMENTS

MS ID	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
CPU	15	5	15	4	5	10	5	11	4	3	8	14	10	4	2
Memory	10	8	26	8	9	12	6	11	6	7	9	16	10	9	2

where $g \sim \text{Gumbel}(0, 1)$ is Gumbel noise, and the temperature parameter τ employs a dynamic annealing strategy that progressively transitions the probability distribution toward a near-one-hot deterministic distribution during later training stages. The policy network is optimized via policy gradient, with its gradient update formula given by:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(a_t|s_t) \cdot A(s_t, a_t)] + \alpha_H \nabla_\theta H(\pi_\theta) \quad (24)$$

where $A(s_t, a_t)$ is the advantage function. The advantage values are normalized to reduce training bias. To control the policy update magnitude, we introduce a trust region constraint by limiting the KL divergence between the old and new policy distributions:

$$L^{KL}(\theta) = \beta_{KL} D_{KL}[\pi_{\theta_{old}} \parallel \pi_\theta] \quad (25)$$

where D_{KL} denotes the KL divergence, and β_{KL} is a dynamically adjusted coefficient.

2) *Critic for State Evaluation*: The value network shares the underlying graph neural network layers with the policy network while maintaining an independent output structure. Specifically, the value network leverages the same graph representation learning process as the policy network and employs a multilayer perceptron (MLP) to map the global system state representation z into the expected return of the current state, denoted as $V_\phi(s_t)$:

$$V_\phi(s_t) = f_{\text{MLP}}(z) \quad (26)$$

The optimization objective of the value network is to minimize the mean squared error between the predicted value and the target value, formulated as:

$$L(\phi) = \mathbb{E}_{s_t} [(V_\phi(s_t) - V^{\text{target}}(s_t))^2] \quad (27)$$

where $V^{\text{target}}(s_t) = r_t + \gamma V_{\phi'}(s_{t+1})$ is the target value calculated based on temporal-difference learning, ϕ' represents the target network parameters. Additionally, we apply gradient clipping techniques to prevent gradient explosion: $\nabla \phi \leftarrow \text{clip}(\nabla \phi, -c, c)$, where c is the gradient clipping threshold.

VII. PERFORMANCE EVALUATION

This section evaluates the proposed SAMD-SRG and compares it with benchmark methods.

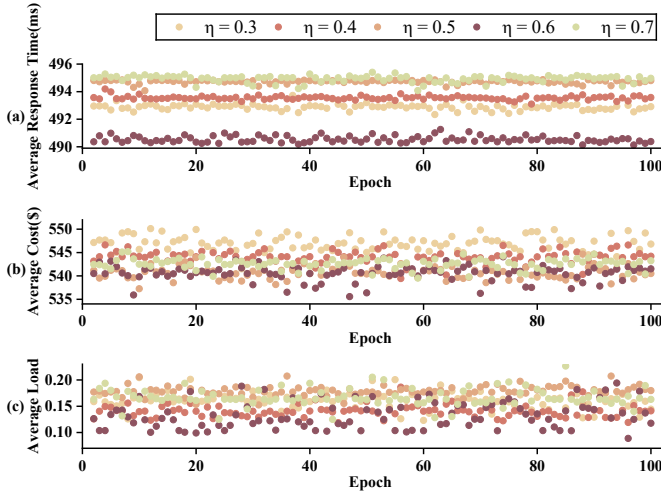


Fig. 7. Performance Comparison for Different Weight Coefficients

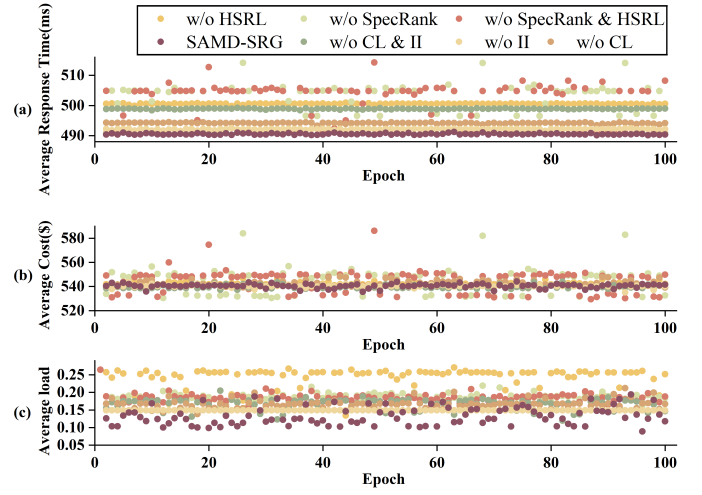


Fig. 8. Performance Comparison of Different Model Configurations

A. Experimental Setup

1) *Simulation Settings*: The experiments utilize the real-world EUA dataset [42] from an Australian Central Business District (CBD) area combined with synthetic data. SAMD-SRG is implemented using PyTorch, with parallel training and evaluation performed on a server equipped with Intel i9-12900K CPUs and NVIDIA RTX 4090 GPUs.

• *Edge-cloud Environment*. The EUA dataset contains geographic locations of 125 edge servers and 817 user devices. A cross-regional benchmark scenario (Scene 1) is constructed by selecting 4 edge computing nodes and 47 mobile users considering geographical regions and user density distributions. Through comprehensive analysis of the longitude and latitude coordinates of edge servers and user devices within the EUA dataset, the selected edge nodes are ensured to maintain geographical diversity and provide coverage across regions with varying user density characteristics.

• *Environment configuration*. Edge servers vary in computing resources, service coverage, and bandwidth constraints, with differentiated resource pricing to simulate real-world economic characteristics (Table III). The temporal variability of user request traffic is incorporated to model realistic load fluctuations over time (Fig. 5).

• *Application Requests and Microservices*. Twenty heterogeneous applications were utilized, each comprising fifteen microservices that exhibited varying resource demands (Table IV). A microservice affinity matrix is employed to simulate dependency relationships between microservices. User requests follow a power-law distribution, where 80% of requests concentrate on four popular applications, exhibiting a bimodal workload pattern.

• *Cross-region Scenes*. Four deployment scenes (Fig. 6) are built using OpenStreetMap, each with distinct network coverage and user distribution: (i) Scene 1 (Baseline): Two large and two small zones with servers along major urban roads. (ii) Scene 2: Hotspot and non-hotspot regions. (iii) Scene 3: Uniform server coverage while uneven user distribution. (iv) Scene 4: Large-scale coverage of a CBD and surrounding

areas.

2) *Benchmark Methods*: For comprehensive evaluation, we compare SAMD-SRG with:

- *Proximal Policy Optimization (PPO)* [43]: A policy-gradient RL method with stable training.
- *First-Fit Decreasing (FFD)* [44]: A classical heuristic algorithm that sorts microservices in descending order of resource demands and deploys each microservice to the first edge node with sufficient available resources.
- *Round-Robin (RR)* [45]: A load-agnostic scheduling strategy that allocates microservices to edge nodes in a fixed cyclic order.
- *Greedy*: Prioritizes deploying microservices to edge nodes with the most abundant available resources.
- *TransEdge* [46]: A GNN-based DRL method employing an efficient task offloading strategy based on a greedy algorithm.
- *GCN-DQN* [47]: A DRL approach that integrates GCN with Deep Q-Networks.

B. Results Analysis

We primarily consider three performance metrics: (i) *Average Response Time* (T_{avg}), defined as the end-to-end latency from request initiation to completion for all user service requests; (ii) *Average Deployment Cost* (C_{avg}), representing the economic expenditure required for service deployment; and (iii) *Average Load* ($\bar{L}$), quantifying the resource utilization equilibrium across heterogeneous edge nodes.

1) *Impact of Weight Coefficient η on SAMD-SRG*: Fig. 7 illustrates the impact of the CPU utilization weighting coefficient η on microservice deployment performance. SAMD-SRG attains optimal overall performance at $\eta=0.6$, exhibiting the lowest average response time (approximately 490 ms), maintaining relatively low deployment costs and minimal average load. This is because the system can prioritize computation-intensive tasks while maintaining reasonable allocation of Memory resources. In contrast, when $\eta=0.7$, the average response time reaches its peak with corresponding increases in server load, indicating that excessive bias toward

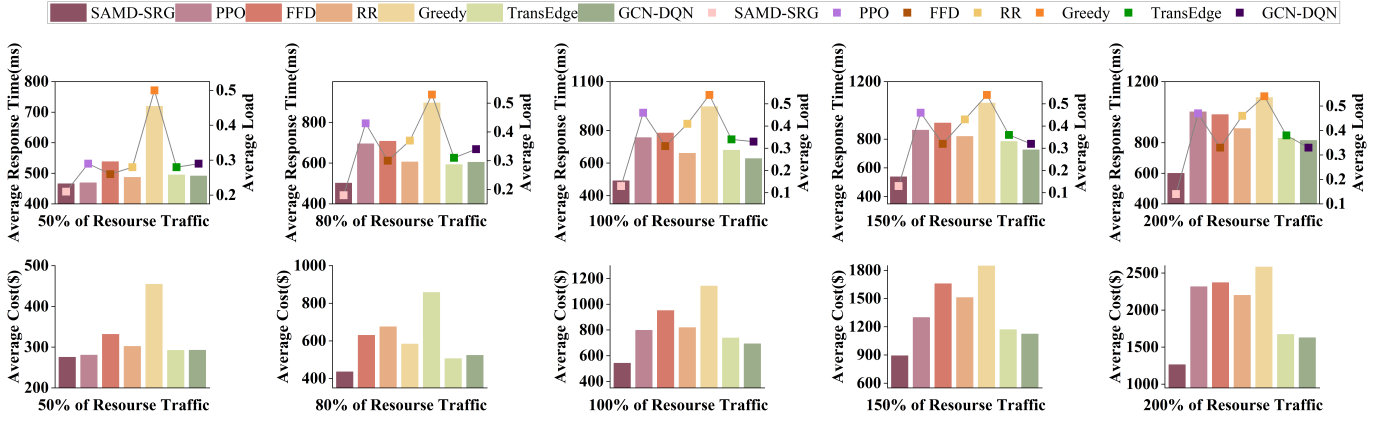


Fig. 9. Performance Comparison Under Dynamic Request Traffic

CPU resource allocation leads to reduced overall efficiency. Conversely, at $\eta=0.3$, deployment costs rise significantly, revealing that over-emphasis on Memory resource allocation increases deployment overhead. Notably, deployment performance under all η configurations remain stable throughout the training process with relatively small fluctuations.

2) *Ablation Study*: As shown in Fig. 8, we evaluated the performance of seven model configurations over 100 training epochs. By comparing 'w/o HSRL' with the complete SAMD-SRG, we observed that the introduction of HSRL reduced the average response time from approximately 500ms to 490ms, and improved load balancing performance by 52%. This indicates that HSRL effectively captures the bidirectional interaction representations of services and edge resources, thereby enabling resource-aware deployment optimization. To further analyze the contribution of key attention modules within HSRL, we evaluated the performance after removing the Cross-Layer Attention (CL) and Information Integration (II) modules. Experiments showed that removing the CL module (w/o CL) led to a more significant performance degradation compared to removing the II module (w/o II), strongly proving CL's critical role in precisely capturing dynamic matching relationships between services and edge nodes. Removing the SpecRank module resulted in a significant increase in average response time and deployment cost, highlighting the role of microservice ordering in reducing the overhead of cross-layer service invocations. The complete removal of both core components led to a marked decline in overall performance. In summary, HSRL and SpecRank contribute prominently to load balancing and response time, respectively. The complete SAMD-SRG, by integrating these two core components, ultimately achieves optimal overall performance.

3) *Comparison with Baselines Over 24 Hours*: Fig. 10 illustrates the 24-hour performance comparison of seven methods. SAMD-SRG demonstrates superior performance across average response time, deployment cost, and average load, particularly excelling during peak traffic periods (10:00-13:00 and 20:00-23:00). At 11:00, SAMD-SRG achieves an average response time of approximately 600ms, while TransEdge exceeds 800ms, a 25% reduction, indicating faster service response for users. From a server load perspective, SAMD-

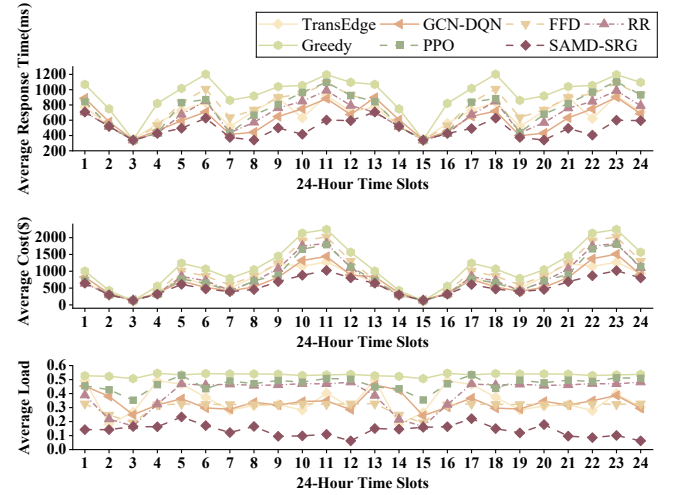


Fig. 10. Performance Comparison Across 24-Hour Time Slots

SRG consistently maintains the lowest average load with minimal fluctuations, demonstrating its exceptional load-balancing capability. This stability is critical for ensuring QoS and optimizing resource utilization, especially under sudden traffic spikes. In contrast, Greedy exhibits the weakest performance in dynamic edge cloud environments. TransEdge and GCN-DQN perform better than the other methods but are outperformed by SAMD-SRG. While all methods reflect the inherent temporal periodicity of real-world workloads, SAMD-SRG effectively adapts to load variations by learning environmental dynamics, even amidst these periodic fluctuations.

4) *Performance Comparison Under Dynamic Request Traffic*: Fig. 9 highlights SAMD-SRG's robust performance across varying request traffic loads, from 50% to 200%. At low load (50%), while SAMD-SRG's overall performance is comparable to the suboptimal PPO, it exhibits notably more balanced server load distribution, indicating superior heterogeneous resource allocation. Under standard load (100%), SAMD-SRG reduces response time and deployment costs by approximately 48% and 52.48% respectively compared to Greedy, while maintaining lower resource load. At 200% load, SAMD-SRG's response time increases by only approximately 22%

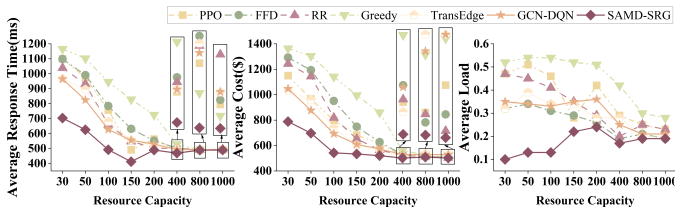


Fig. 11. Performance Comparison Under Resource Capacity Settings

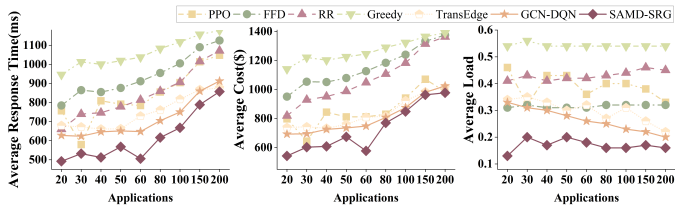


Fig. 12. Performance Comparison Under Application Numbers

compared to the 100% load, a significant improvement over the approximately 30% rise in the sub-optimal GCN-DQN, indicating its stability. Moreover, SAMD-SRG's deployment cost is approximately 22% less than GCN-DQN's, and its sustained lower resource load across all tests is beneficial for long-term operation.

5) *Performance Comparison Under Varying Resource Capacities:* As illustrated in Fig. 11, our approach demonstrates superior performance across the resource capacity range of 30% to 1000%. Under the severely constrained condition of 30% capacity, SAMD-SRG achieves reductions of 27%, 23%, and 68% in response time, deployment cost, and average load, respectively, compared to the sub-optimal TransEdge. This advantage is primarily attributed to SAMD-SRG's SpecRank-based intelligent prioritization mechanism for critical service resource allocation, which optimizes overall performance under resource constraints. With the increase in resource capacity, the performance gap between SAMD-SRG and other methods tends to converge, indicating that abundant resources alleviate the difficulty of optimization. However, SAMD-SRG maintains robust performance, owing to its integrated graph neural network-driven representation learning and reinforcement learning-driven decision optimization, thereby enabling efficient service deployment.

6) *Impact of Application Numbers on Performance:* Fig. 12 illustrates performance across 20 to 200 applications. Generally, response time and deployment cost increase with application scale for all methods. However, traditional approaches like FFD, RR, and Greedy exhibit a notably accelerated increase in these metrics beyond 60 applications, indicating limitations for large-scale deployments. For instance, the response time of RR increases sharply from approximately 800ms to over 1000ms, and its deployment cost rises from \$1050 to \$1360, while maintaining a high average load above 0.40. Although TransEdge and GCN-DQN show better performance than these traditional methods in some aspects, their overall performance is inferior to SAMD-SRG, particularly in load control. In contrast, SAMD-SRG consistently achieves the lowest average deployment cost across all test scenarios, indicating its higher

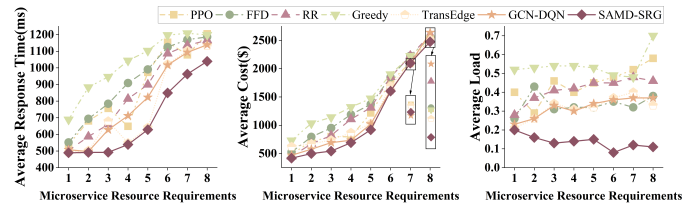


Fig. 13. Performance Comparison Under Different Resource Requirements

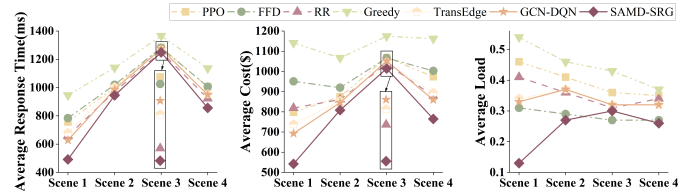


Fig. 14. Performance Comparison Across Four Cross-Region Scenes

resource utilization efficiency, and maintains a stable and low average load (0.1-0.2), which demonstrates its superior load balancing capability.

7) Impact of Resource Requirements on Performance:

Fig. 13 presents a comparative performance evaluation of seven approaches across varying microservice resource demand levels (50%-800%). As resource demands increase, traditional methods (e.g., Greedy and RR) exhibit nearly linear growth in response time and deployment costs, whereas SAMD-SRG shows more gradual performance metric escalation, with average load even displaying a downward trend. This highlights its scalability in resource-intensive scenes. At the peak resource demand level of 800%, while the deployment costs across all methods converge, SAMD-SRG achieves a maximum reduction of 13.76% in average response time and sustains a stable average load around 0.10. This advantage stems from SAMD-SRG's prioritization of co-locating high-resource-consuming microservices with their frequently interacting counterparts, thereby reducing cross-node communication overhead.

8) Performance Comparison Across Four Cross-Region Scenes:

Fig. 14 compares method performance across four cross-region deployment scenes. SAMD-SRG achieves particularly superior performance in Scene 1, with all key metrics significantly outperforming the baselines. Notably, all methods exhibit performance degradation in the other scenarios, highlighting the challenges that large-scale dynamic request loads pose to microservice deployment. Despite Scene 4's larger scale compared to Scene 3, SAMD-SRG demonstrates remarkable performance improvement, even surpassing Scene 2. This suggests that SAMD-SRG exhibits enhanced capability in global resource scheduling optimization for large-scale deployments. The Greedy consistently yields the poorest performance across all scenes, confirming that simplistic heuristic strategies lack the necessary resource awareness and optimization mechanisms required for complex cross-region microservice deployment.

VIII. CONCLUSION AND FUTURE WORK

We proposed SAMD-SRG, a novel microservice deployment framework that integrated spectral clustering and graph reinforcement learning for edge-cloud environments. The framework incorporated SpecRank for intelligent microservice grouping through Hermitian spectral clustering, alongside HSRL, which leveraged graph convolutional networks with cross-layer attention mechanisms to model service-edge interactions. Furthermore, we formalized the microservice deployment problem as a Markov Decision Process and designed GEAC to optimize deployment decisions through shared graph representation learning. Comprehensive evaluations demonstrated that SAMD-SRG achieved significant improvements over existing baselines, including a 48% reduction in response time, 52.5% lower deployment costs, and 75.93% improvement in load balancing across benchmark scenarios. Future research will focus on enhancing microservice ranking methodologies by incorporating data transmission and developing dynamic scoring mechanisms that adapt to evolving service dependencies in real-time environments.

REFERENCES

- [1] A. M. Maia and Y. Ghamri-Doudane, "A deep reinforcement learning approach for the placement of scalable microservices in the edge-to-cloud continuum," in *GLOBECOM 2023-2023 IEEE Global Communications Conference*. IEEE, 2023, pp. 479–485.
- [2] A. M. Rahmani, A. Haider, P. Khoshvaght, F. S. Gharehchopogh, K. Moghaddasi, S. Rajabi, and M. Hosseinzadeh, "Optimizing task offloading with metaheuristic algorithms across cloud, fog, and edge computing networks: A comprehensive survey and state-of-the-art schemes," *Sustainable Computing: Informatics and Systems*, p. 101080, 2025.
- [3] W. Hua, P. Liu, and L. Huang, "Energy-efficient resource allocation for heterogeneous edge-cloud computing," *IEEE Internet of Things Journal*, vol. 11, no. 2, pp. 2808–2818, 2023.
- [4] S. Deng, Z. Xiang, J. Taheri, M. A. Khoshkholghi, J. Yin, A. Y. Zomaya, and S. Dustdar, "Optimal application deployment in resource constrained distributed edges," *IEEE transactions on mobile computing*, vol. 20, no. 5, pp. 1907–1923, 2020.
- [5] S. Luo, H. Xu, C. Lu, K. Ye, G. Xu, L. Zhang, J. He, and C. Xu, "An in-depth study of microservice call graph and runtime performance," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 12, pp. 3901–3914, 2022.
- [6] C. Sun, X. Li, C. Wang, Q. He, X. Wang, and V. C. Leung, "Hierarchical deep reinforcement learning for joint service caching and computation offloading in mobile edge-cloud computing," *IEEE Transactions on Services Computing*, vol. 17, no. 4, pp. 1548–1564, 2024.
- [7] K. Peng, Y. Hu, H. Ding, H. Chen, L. Wang, C. Cai, and M. Hu, "Large-scale service mesh orchestration with probabilistic routing in cloud data centers," *IEEE Transactions on Services Computing*, 2025.
- [8] B. Tang, F. Guo, B. Cao, M. Tang, and K. Li, "Cost-aware deployment of microservices for iot applications in mobile edge computing environment," *IEEE Transactions on Network and Service Management*, vol. 20, no. 3, pp. 3119–3134, 2022.
- [9] M. Liu, Z. Tu, H. Xu, X. Xu, and Z. Wang, "Dysr: A dynamic graph neural network based service bundle recommendation model for mashup creation," *IEEE Transactions on Services Computing*, vol. 16, no. 4, pp. 2592–2605, 2023.
- [10] W. Lv, P. Yang, T. Zheng, C. Lin, Z. Wang, M. Deng, and Q. Wang, "Graph-reinforcement-learning-based dependency-aware microservice deployment in edge computing," *IEEE Internet of Things Journal*, vol. 11, no. 1, pp. 1604–1615, 2023.
- [11] B. Hu and Z. Cao, "Minimizing resource consumption cost of dag applications with reliability requirement on heterogeneous processor systems," *IEEE Transactions on Industrial Informatics*, vol. 16, no. 12, pp. 7437–7447, 2019.
- [12] L. Qian, J. Li, X. He, R. Gu, J. Shao, and Y. Lu, "Microservice extraction using graph deep clustering based on dual view fusion," *Information and Software Technology*, vol. 158, p. 107171, 2023.
- [13] J. Zhang, G. Zhang, X. Bao, C. Ding, P. Yuan, X. Zhang, and S. Wang, "Dependent application offloading in edge computing," *IEEE Transactions on Cloud Computing*, vol. 11, no. 4, pp. 3439–3451, 2023.
- [14] Y. Zhang, W. Hua, Z. Zhou, G. E. Suh, and C. Delimitrou, "Sinan: ML-based and qos-aware resource management for cloud microservices," in *Proceedings of the 26th ACM international conference on architectural support for programming languages and operating systems*, 2021, pp. 167–181.
- [15] X. He, Z. Tu, M. Wagner, X. Xu, and Z. Wang, "Online deployment algorithms for microservice systems with complex dependencies," *IEEE Transactions on Cloud Computing*, vol. 11, no. 2, pp. 1746–1763, 2022.
- [16] A. Mirhosseini, S. Elnikety, and T. F. Wenisch, "Parslo: A gradient descent-based approach for near-optimal partial slo allotment in microservices," in *Proceedings of the ACM Symposium on Cloud Computing*, 2021, pp. 442–457.
- [17] F. Guo, B. Tang, and M. Tang, "Joint optimization of delay and cost for microservice composition in mobile edge computing," *World Wide Web*, vol. 25, no. 5, pp. 2019–2047, 2022.
- [18] K. Fu, W. Zhang, Q. Chen, D. Zeng, and M. Guo, "Adaptive resource efficient microservice deployment in cloud-edge continuum," *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 8, pp. 1825–1840, 2021.
- [19] W. Fan, L. Zhao, X. Liu, Y. Su, S. Li, F. Wu, and Y. Liu, "Collaborative service placement, task scheduling, and resource allocation for task offloading with edge-cloud cooperation," *IEEE Transactions on Mobile Computing*, vol. 23, no. 1, pp. 238–256, 2022.
- [20] F. Guo, B. Tang, M. Tang, and W. Liang, "Deep reinforcement learning-based microservice selection in mobile edge computing," *Cluster Computing*, vol. 26, no. 2, pp. 1319–1335, 2023.
- [21] K. Gasmli, K. Abassi, L. Romdhani, and O. Debauche, "Reinforcement learning-based approach for microservices-based application placement in edge environment," in *2023 IEEE Symposium on Computers and Communications (ISCC)*. IEEE, 2023, pp. 1098–1103.
- [22] Y. Jia, Z. Huang, J. Yan, Y. Zhang, K. Luo, and W. Wen, "Joint optimization of resource allocation and data selection for fast and cost-efficient federated edge learning," *IEEE Transactions on Cognitive Communications and Networking*, 2024.
- [23] T. Lyu, H. Xu, F. Liu, M. Li, L. Li, and Z. Han, "Two layer stackelberg game-based resource allocation in cloud-network convergence service computing," *IEEE Transactions on Cognitive Communications and Networking*, 2024.
- [24] M. Zambianco, S. Cretti, and D. Siracusa, "Disruption-aware microservice re-orchestration for cost-efficient multi-cloud deployments," *arXiv preprint arXiv:2501.16143*, 2025.
- [25] A. Mohammadzadeh, M. Masdari, and F. S. Gharehchopogh, "Energy and cost-aware workflow scheduling in cloud computing data centers using a multi-objective optimization algorithm," *Journal of Network and Systems Management*, vol. 29, no. 3, p. 31, 2021.
- [26] H. Zeng, Z. Zhu, Y. Wang, Z. Xiang, and H. Gao, "Periodic collaboration and real-time dispatch using an actor-critic framework for uav movement in mobile edge computing," *IEEE Internet of Things Journal*, vol. 11, no. 12, pp. 21 215–21 226, 2024.
- [27] D. Garg, N. C. Narendra, and S. Tesfatsion, "Heuristic and reinforcement learning algorithms for dynamic service placement on mobile edge cloud," *arXiv preprint arXiv:2111.00240*, 2021.
- [28] L. Gu, Z. Chen, H. Xu, D. Zeng, B. Li, and H. Jin, "Layer-aware collaborative microservice deployment toward maximal edge throughput," in *IEEE INFOCOM 2022-IEEE Conference on Computer Communications*. IEEE, 2022, pp. 71–79.
- [29] A. Zhou, S. Wang, S. Wan, and L. Qi, "Lmm: latency-aware microservice mashup in mobile edge computing environment," *Neural Computing and Applications*, vol. 32, no. 19, pp. 15 411–15 425, 2020.
- [30] W. Ma, R. Wang, Y. Gu, Q. Meng, H. Huang, S. Deng, and Y. Wu, "Multi-objective microservice deployment optimization via a knowledge-driven evolutionary algorithm," *Complex & Intelligent Systems*, vol. 7, pp. 1153–1171, 2021.
- [31] A. M. Rahmani, J. Tanveer, F. S. Gharehchopogh, S. Rajabi, and M. Hosseinzadeh, "A novel offloading strategy for multi-user optimization in blockchain-enabled mobile edge computing networks for improved internet of things performance," *Computers and Electrical Engineering*, vol. 119, p. 109514, 2024.
- [32] J. Lu, J. Yang, S. Li, Y. Li, W. Jiang, J. Dai, and J. Hu, "A2c-drl: Dynamic scheduling for stochastic edge-cloud environments using a2c and deep reinforcement learning," *IEEE Internet of Things Journal*, 2024.
- [33] X. Zhou, J. Yang, Y. Li, S. Li, and Z. Su, "Deep reinforcement learning-based resource scheduling for energy optimization and load balancing

in sdn-driven edge computing,” *Computer Communications*, vol. 226, p. 107925, 2024.

- [34] X. Zhou, J. Yang, Y. Li, S. Li, Z. Su, and J. Lu, “Ec-trl: Evolutionary-weighted clustering and transformer-augmented reinforcement learning for dynamic resource scheduling in edge cloud environments,” *IEEE Internet of Things Journal*, 2024.
- [35] J. Yang, J. Lu, X. Zhou, S. Li, C. Xiong, and J. Hu, “Ha-a2c: Hard attention and advantage actor-critic for addressing latency optimization in edge computing,” *IEEE Transactions on Green Communications and Networking*, 2024.
- [36] A. M. Rahmani, A. Haider, S. Ali, S. Rajabi, F. S. Gharehchopogh, P. Khoshvaght, and M. Hosseinzadeh, “An optimizing geo-distributed edge layering with double deep q-networks for predictive mobility-aware offloading in mobile edge computing,” *Ad Hoc Networks*, vol. 172, p. 103804, 2025.
- [37] K. Moghaddasi, S. Rajabi, F. S. Gharehchopogh, and A. Ghaffari, “An advanced deep reinforcement learning algorithm for three-layer d2d-edge-cloud computing architecture for efficient task offloading in the internet of things,” *Sustainable Computing: Informatics and Systems*, vol. 43, p. 100992, 2024.
- [38] H. Gao, X. Wang, W. Wei, A. Al-Dulaimi, and Y. Xu, “Com-ddpg: Task offloading based on multiagent reinforcement learning for information-communication-enhanced mobile edge computing in the internet of vehicles,” *IEEE Transactions on Vehicular Technology*, vol. 73, no. 1, pp. 348–361, 2024.
- [39] A. Xu, Z. Hu, X. Li, R. Tian, X. Zhang, B. Chen, H. Xiao, H. Zheng, X. Feng, M. Zheng *et al.*, “Transedge: Task offloading with gnn and drl in edge computing-enabled transportation systems,” *IEEE Internet of Things Journal*, 2024.
- [40] M. S. H. Chy, K. Sooksatra, J. Yero, and T. Cerny, “Benchmarking micro2micro transformation: an approach with gnn and vae,” *Cluster Computing*, vol. 27, no. 4, pp. 4171–4185, 2024.
- [41] Y. Xu, Z. Qiu, H. Gao, X. Zhao, L. Wang, and R. Li, “Heterogeneous data-driven failure diagnosis for microservice-based industrial clouds toward consumer digital ecosystems,” *IEEE Transactions on Consumer Electronics*, vol. 70, no. 1, pp. 2027–2037, 2024.
- [42] G. Cui, Q. He, F. Chen, H. Jin, Y. Xiang, and Y. Yang, “Location privacy protection via delocalization in 5g mobile edge computing environment,” *IEEE Transactions on Services Computing*, vol. 16, no. 1, pp. 412–423, 2021.
- [43] W. Zhao, C. Wu, R. Zhong, K. Shi, and X. Xu, “Edge computing and caching optimization based on ppo for task offloading in rsu-assisted iov,” in *2023 IEEE 9th World Forum on Internet of Things (WF-IoT)*. IEEE, 2023, pp. 01–06.
- [44] S. Long, W. Wen, Z. Li, K. Li, R. Yu, and J. Zhu, “A global cost-aware container scheduling strategy in cloud data centers,” *IEEE Transactions on Parallel and Distributed Systems*, vol. 33, no. 11, pp. 2752–2766, 2021.
- [45] L. Hamid, A. Jadoon, and H. Asghar, “Comparative analysis of task level heuristic scheduling algorithms in cloud computing,” *The Journal of Supercomputing*, vol. 78, no. 11, pp. 12 931–12 949, 2022.
- [46] A. Xu, Z. Hu, X. Li, R. Tian, X. Zhang, B. Chen, H. Xiao, H. Zheng, X. Feng, M. Zheng *et al.*, “Transedge: Task offloading with gnn and drl in edge computing-enabled transportation systems,” *IEEE Internet of Things Journal*, 2024.
- [47] M. Du, X. Sun, W. Lin, and W. Zhan, “Deep reinforcement learning-based access point selection for cell-free massive mimo with graph convolutional networks,” in *2023 International Conference on Future Communications and Networks (FCN)*. IEEE, 2023, pp. 1–5.



Keli Liu received the B.S. degree in School of Software from Henan Finance University, Zhengzhou, China, in 2022. She is currently pursuing the M.S. degree with the State Key Laboratory of Public Big Data, Guizhou University, Guiyang, China. Her research interests include edge computing, resource scheduling.



Jing Yang (Member, IEEE) received the PhD degree in mechanical and electronic engineering from Guizhou University, in 2020. From August 2018 to September 2019, he was awarded a scholarship by the China Scholarship Council (CSC) under the State Scholarship Fund to pursue his study with Oklahoma State University, as a Joint Ph.D. student with the school of Computer Science and Technology, where he joined the Guoliang Fan’s Group. From October 2022 to October 2025, he was a visiting scholar studying in the team of Wu Fan’s (NSFC Excellent Young Scholars) and a postdoctoral in Professor Guo Minyi (IEEE Fellow) from Shanghai Jiao Tong University. He is currently an assistant professor with the State Key Laboratory of Public Big Data, Guizhou University, China. His research interests include high performance computing, task scheduling in various architectures and Open domain visual learning. He has published more than 50 peer-reviewed papers in the related area, including well-archived international journals such as IEEE Transactions on Circuits and Systems for Video Technology, IEEE Transactions on Green Communications and Networking, IEEE Transactions on Intelligent transportation systems, and IEEE Internet of Things Journal. He serves more than 20 prestigious international journals and conferences, such as IEEE TC, IEEE TMC, IEEE TI, IEEE TIM, IEEE TSC, IEEE TSM, IEEE TAI, IEEE TASE, IEEE TOJ, IEEE TCE, IEEE TMM, IEEE TCSVT, IEEE TNLS, IEEE TIFS, IEEE TPDS.



Xiaoli Ruan received the PhD degree in information and communication engineering from Yunnan University, in 2021. Participating in the “Jiaxiu Zhiguang” Visiting Scholar Program at Zhejiang University from September 2024 to June 2025. She is currently a lecturer with the State Key Laboratory of Public Big Data, Guizhou University, China. Her research interests include Artificial Intelligence and Multimodal Big Data Fusion. She has published more than 20 peer-reviewed papers in the related area, including well-archived international journals such as IEEE Transactions on Geoscience and Remote Sensing, Knowledge-Based Systems, and IEEE Internet of Things Journal.



Han Zhao received the Ph.D. degree from the Department of Computer Science and Engineering, Shanghai Jiao Tong University, China, in June 2022. He is an Assistant Professor with the Department of Computer Science and Engineering, Shanghai Jiao Tong University, China. His research interests include high performance computing and resource management of accelerators in datacenters.



Shixuan Sun received the Ph.D. in Computer Sciences from the Department of Computer Science and Engineering, Hong Kong University of Science and Technology (HKUST) in 2020, supervised by Prof. Qiong Luo. Prior to that, he got his M.S and B.S. in Computer Sciences from the School of Software Engineering, Tongji University in 2014 and 2011 respectively. From 2020 to 2023, he worked as a research fellow at the School of Computing, National University of Singapore, supervised by Prof. Bingsheng He. He is currently a Tenure-Track

Associate Professor at the Department of Computer Science and Engineering, Shanghai Jiao Tong University. His research interests are in database systems, graph algorithms, and parallel computing.



Shaobo Li received the B.E. degree in machinery manufacturing and automation from the Huazhong University of Science and Technology, Wuhan, China, in 1996, the M.E. degree in mechanical engineering from the Guizhou University of Technology, Guiyang, China, in 1999, and the Ph.D. degree in computer software and theory from the Chinese Academy of Sciences, Chengdu, China, in 2003. He is currently the Director of the State Key Laboratory of Public Big Data, Guizhou University, and a Professor of the Key Laboratory of Modern

Manufacturing Technology of the Ministry of Education and the College of Mechanical Engineering. His research interests include public Big Data fusion and integration, and manufacturing Big Data.



Minyi Guo (Fellow, IEEE) received the B.S. and M.E. degrees in computer science from Nanjing University, China, and the PhD degree in computer science from the University of Tsukuba, Japan. Additionally, he is the Department of Computer Science and Engineering director of the Embedded and Pervasive Computing Center, Shanghai Jiao Tong University. Before joining SJTU, he had been a professor with the School of Computer Science and Engineering, University of Aizu, Japan. His present research interests include parallel/distributed

computing, compiler optimizations, Big Data, and cloud computing. He has more than 400 publications in major journals and international conferences in these areas. He received 7 best/highlight paper awards from international conferences including ALSPOS 2017 and ICCD 2018. He is now editor-in-chief of IEEE Transactions on Sustainable Computing, and on the editorial board of IEEE Transactions on Parallel and Distributed Systems, IEEE Transactions on Cloud Computing, and Journal of Parallel and Distributed Computing. He is the Zhiyuan chair professor, and an ACM distinguished member.